Technical Description of the Evid Codebase

Introduction

The "evid" codebase implements an evidence management tool designed for handling PDF and text documents, facilitating the addition, labeling, and organization of evidence for research or rebuttal purposes. It supports generating BibTeX files, creating Typst-based labels, and producing rebuttal documents. The tool integrates a command-line interface (CLI) and a graphical user interface (GUI) built with PySide6, ensuring usability across different environments.

Architecture Overview

The codebase is structured in Python, organized under the `src/evid/` directory, with a modular design to separate concerns:

- `src/evid/cli/`: Contains modules for command-line operations, including dataset management, evidence addition, labeling, and BibTeX generation.
- `src/evid/core/`: Houses core business logic, such as label creation, BibTeX processing, PDF metadata extraction, and rebuttal document generation.
- `src/evid/gui/`: Implements the graphical interface using PySide6, with tabs for adding and browsing evidence.
- `src/evid/utils/`: Provides utility functions, including text normalization.
- `src/evid/__init__.py`: Initializes the package, loads configuration, and defines constants.

The system uses Pydantic for data validation and models, ensuring type safety and structured data handling. Logging is managed via the Rich library for enhanced console output.

Core Components

Command-Line Interface (CLI)

The CLI, defined in `src/evid/cli/main.py`, uses the `treeparse` library for argument parsing and command structuring. It supports commands for:

- Dataset operations: Creating, listing, and tracking datasets with Git.
- Evidence management: Adding PDFs or URLs, listing documents, labeling, and generating BibTeX.
- Configuration: Updating and showing settings from `~/.evidrc`.
- Rebuttal: Creating rebuttal documents from labeled evidence.

Options include dataset selection, file paths, and flags for autolabeling or labeling.

Core Logic

- `src/evid/core/models.py`: Defines Pydantic models for configuration (`ConfigModel`) and document metadata (`InfoModel`).
- `src/evid/core/label.py`: Handles label file creation and editor integration.

- `src/evid/core/label_setup.py`: Processes text and PDF content to generate Typst files for labeling.
- `src/evid/core/bibtex.py`: Generates BibTeX from Typst query outputs.
- `src/evid/core/pdf_metadata.py`: Extracts metadata from PDFs using `pypdf`.
- `src/evid/core/rebut_doc.py`: Produces rebuttal documents by parsing BibTeX and integrating notes.
- `src/evid/core/database.py`: Manages a simple database of documents using YAML files.

Graphical User Interface (GUI)

Implemented in `src/evid/gui/main.py`, the GUI features a tabbed interface:

- Add Evidence Tab: Allows users to select datasets, input metadata, and add PDFs or URLs.
- Browse Evidence Tab: Displays a table of documents with search, selection, and actions for labeling, BibTeX generation, and rebuttal.

It enforces a dark theme for consistency and includes keyboard shortcuts for navigation.

Utilities

- `src/evid/utils/text.py`: Normalizes text to UTF-8, handling Danish characters and encoding issues.

Functionality and Data Flow

1. **Configuration**: Loaded from `~/.evidrc` or defaults, using `ConfigModel`.
2. **Dataset Management**: Directories under the default path, optionally tracked with Git.
3. **Evidence Addition**: Documents are hashed for uniqueness, stored in UUID-based directories with metadata in `info.yml`.
4. **Labeling**: Generates `label.typ` from PDFs/text, opens in editor, and produces `label.bib` via Typst queries.
5. **BibTeX and Rebuttal**: Parses labels to create BibTeX, then generates rebuttal Typst files incorporating notes.
6. **GUI Interaction**: Provides visual access to CLI features, with real-time updates.

Key features include URL support for PDFs, autolabeling for paragraphs, and integration with external tools like Typst and Git.

Dependencies

- Core: `pathlib`, `yaml`, `requests`, `beautifulsoup4`, `pypdf`, `fitz` (PyMuPDF), `arrow`, `hashlib`, `uuid`.
- CLI/GUI: `treeparse`, `rich`, `PySide6`, `subprocess`.
- Validation: `pydantic`.
- Optional: `GitPython` for dataset tracking.

Conclusion

The evid codebase is a robust, modular system for evidence management, combining CLI efficiency with GUI accessibility. Its design supports extensibility, with clear separation of concerns and strong validation mechanisms, making it suitable for academic or professional document handling.